Workshop 3: Plotting

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn
NHH Norwegian School of Economics

October 16, 2026

See GitHub repository for notebooks and data:

https://github.com/richardfoltyn/TECH2-H26

Contents

1 Exercise 1: Plotting functions 1

2 Exercise 2: Visualizing US business cycles and recessions 2

3 Exercise 3: Daily returns of US stock market indices 2

4 Exercise 4: Population and electric cars in Norway 3

5 Exercise 5: Cruise ship arrivals in Bergen 4

Exercise 1: Plotting functions

Consider the following function in one variable x,

$$f(x) = \sin(20x) + 5x^2$$

which is the sum of the sine function and a quadratic function.

Perform the following tasks:

1. Define a Python function that returns the function value $f(x)$ for some argument x. The function should work with both scalar numbers and NumPy arrays.

   Hint: The sine function is implemented in `np.sin()`.

2. Evaluate the function on a uniformly spaced grid of x-values on the interval $[-1, 1]$ using 51 grid points.

   Create a plot depicting the function on this interval. Set the x-ticks to the values $[-1, 0, 1]$ and add a legend.

3. Locate the point x_{min} where the function attains its minimum on the grid you created and print the values $(x_{min}, f(x_{min}))$.

   Hint: You can use the NumPy function `np.argmin()` for this.

4. Augment the plot you created earlier with a horizontal and vertical line indicating where the minimum is located. Add a black dot at the point $(x_{min}, f(x_{min}))$.

   Hint: Horizontal and vertical lines can be added with `axhline()` and `axvline()`. A single dot can be added in various ways, e.g., using `scatter()`.

5. Repeat the exercise but now use $N = 501$ grid points to evaluate the function.

Exercise 2: Visualizing US business cycles and recessions

In this exercise, you are asked to plot two main business cycle indicators, GDP growth and the unemployment rate, for the United States.

1. Load the data from the CSV file `FRED/FRED_quarterly.csv` located in the repository's `data` folder.

   Familiarize yourself with the data by printing the columns present in the data set and the first 5 rows of data.

   Hint: Use `read_csv()` with `parse_dates=['DATE']` and `index_col='DATE'` to automatically parse strings stored in the `DATE` column as dates and set `DATE` as the index.

2. Compute the quarterly GDP growth (i.e., the relative change of the column `GDP` vs. the previous quarter) in percent.

   Hint: This can be achieved using the `pct_change()` method.

3. Create a figure with two subplots (arranged in 2 rows) which show the GDP growth and the unemployment rate.

   Hint: You can construct the graph either using Matplotlib's plotting function or using pandas's `DataFrame.plot()` method.

4. You want to understand how the business cycle indicators behave around recession starts in the US. The National Bureau of Economic Research (NBER) is a research organization that dates US business cycles.

   The `peak` column records the month immediately before each recession starts.

   1. Load the business cycle dates from `NBER_cycle_dates.csv` located in the `data/` folder.

      Drop all rows where the `peak` date is before 1948.

      Hint: Use `pd.read_csv(..., parse_dates=['peak', 'trough'])` to automatically parse strings in the `peak` and `trough` columns.

   2. Loop through all remaining business cycle peak dates and add a vertical line to each of the two subplots for GDP growth and unemployment at each date. These lines mark the months immediately before recession starts.

   3. Can you detect any patterns for GDP growth and the unemployment rate around recession starts?

Exercise 3: Daily returns of US stock market indices

In this exercise, we examine how the three major US stock market indices performed in 2025. Use the data in `indices.csv` from the folder `../../data/stockmarket` to solve the following tasks:

1. Load the CSV data and restrict the sample to the calendar year 2025.

   Hint: The data is tab-separated, so you need to specify `sep='\t'` when reading in the data.

   Hint: You should use `read_csv()` with `parse_dates=['Date']` to automatically parse strings stored in the `Date` column as dates.

2. The data comes in a "long" format where each date-ticker combination is stored in a separate row. For the following analysis, the data needs to be reshaped such that each ticker symbol is in its own column ("wide" format).

   Use the `pivot()` method to reshape the `DataFrame` into the desired form. Consult the user guide to see a graphical illustration.

   Your pivoted `DataFrame` should look as follows:

Ticker	DJIA	NASDAQ	SP500
Date			
2025-01-02	42392.3	19280.8	5868.5
2025-01-03	42732.1	19621.7	5942.5

3. Create a line plot which shows the time series for each of the three ticker symbols in a single graph, for example using `DataFrame.plot()`.
4. The graph you just created is not very satisfactory as the three indices are recorded on vastly different scales. Express each index relative to its initial value in 2025 (so that all three start with the value 1.0) and recreate the previous graph with this normalized data.
5. Another way to check whether the three indices are co-moving is to compute and plot their daily returns. For each index, compute the daily returns, i.e., the relative change vs. the previous closing price in percent (e.g., using `pct_change()`), and plot the three time series of daily returns in a single graph.
6. The previous plots suggest that the three indices co-move a lot.
   1. In order to quantify the strength of this co-movement, compute the pairwise correlations of daily returns using `corr()`.
   2. Create a figure with 3-by-3 subplots where each off-diagonal plot shows the scatter plot for two of the indices.

      Hint: You can use the function `scatter_matrix()` to accomplish this task. Alternatively, you can create a figure with 3-by-3 subplots using Matplotlib's `subplots()`, iterate over all rows and columns and add a `scatter()` plot to each `axes` object.
   3. Add the correlation coefficient for each stock market index pair as text to the corresponding subplot in the figure you created (this can be done using `text()`).

Exercise 4: Population and electric cars in Norway

How do Norwegian municipalities differ in population size, population density, and the use of electric cars? Use the data in `../../data/ssb/municipalities.csv` to explore these questions.

The file contains one row per municipality. The variables needed here are:

- `municipality`: Municipality name.
- `population`: Number of residents.
- `land_area`: Land area in square kilometers, excluding freshwater.
- `cars`: Number of registered passenger cars, including electric cars.
- `electric_cars`: Number of electric passenger cars, excluding hybrids.

The data come from Statistics Norway (SSB); see the data description for details.

1. Load the variables listed above into a `DataFrame`. Display the first five rows and count the municipalities. Are any of the population, land area, or car counts missing?

   Hint: Use the `usecols` argument of `pd.read_csv()` to select columns, and `DataFrame.info()` with `show_counts=True` to inspect the number of nonmissing observations.

2. Create a new column `pop_density` containing population divided by land area. This measures residents per square kilometer.

   Create a figure with two histograms arranged in one row: population size and population density. Use 30 bins in each histogram. Express population in thousands and label both axes.

   Hint: Use `plt.subplots(1, 2)` and the `hist()` method of each `Axes` object. The height of each bar should show the number of municipalities in that bin.

3. Compute the percentage of passenger cars that are electric in each municipality as `100 * electric_cars / cars`, and store it in a new column `ev_share`.

   Create a second figure with two histograms arranged in one row: the number of electric cars (in thousands) and the EV share (in percent). Again, use 30 bins and label both axes.

   How do these distributions differ? Why might the EV share be more useful than the number of electric cars when comparing municipalities?

4. Create a scatter plot with population density on the horizontal axis and the EV share (in percent) on the vertical axis. Use one equally sized point per municipality and some transparency to make overlapping points easier to see.

   Describe the relationship between population density and the EV share.

5. **[Advanced]** Recreate the scatter plot with a logarithmic horizontal axis. Keep the EV share on its original percentage scale.

   Hint: Use `set_xscale()` with `'log'`. Equal distances on this axis represent equal ratios: moving from 1 to 10 residents per square kilometer takes the same space as moving from 10 to 100.

   Why is this plot easier to read?

Exercise 5: Cruise ship arrivals in Bergen

How do cruise ship arrivals in Bergen vary over time and across seasons? Use `kystverket/bergen_monthly_calls.csv` in the repository's `data/` folder to explore these questions.

The file contains monthly arrival counts by vessel type. The variables needed here are:

- `month`: Arrival month, stored as `YYYY-MM`.
- `vessel_type`: Detailed vessel type.
- `calls`: Monthly aggregate number of reported arrivals, including repeat visits by the same ship.

The cruise category also includes some coastal service vessels. The data come from Kystverket; see the data description for details.

1. Load the CSV file into a `DataFrame`. Display the first five rows and determine the sample period.

   Hint: Use `pd.read_csv()` with `parse_dates=['month']` and `date_format='%Y-%m'` to parse the dates. Each month will be represented by its first day.

2. Select observations with `vessel_type` equal to `'Passenger/Cruise'`. Restrict the sample to the complete calendar years **2013–2025**. Set `month` as the index and sort chronologically. Use this selection for all remaining parts.

   Hint: After parsing `month` as a datetime column, you can compare it directly with date strings. For example, `df_calls['month'] >= '2013-01-01'` selects observations on or after January 1, 2013. Combine two comparisons to select the required date range.

3. Create a line plot of monthly cruise ship arrivals over the selected sample. Label both axes and add a title. Find the month with the most arrivals, print the month and its arrival count, and indicate the month with a vertical line in the plot. Describe the seasonal pattern and the changes visible around 2020–2021.

   Hint: The `idxmax()` method returns the index label of the first maximum.

4. Compute the average number of arrivals for each calendar month using a loop. For each month from January through December, select the corresponding observations, compute the mean of `calls`, and append it to a list.

   Create a bar chart of the 12 averages, with month names on the horizontal axis. Which months have the most arrivals on average?

   Hint: Loop over `range(1, 13)`. The `df_cruise.index.month` attribute gives the calendar month (1 for January through 12 for December) of each observation.